TD/TP1: Complexités algorithmiques

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

23 septembre 2025

Dans ce premier TD/TP, nous étudions la complexité des algorithmes. Ce sujet proposera donc deux types de questions. D'abord des questions de type TD, où vous allez devoir calculer des complexités algorithmiques via les méthodes vues en cours, à partir de fonctions ou de pseudo-codes qui vous seront fournis. Ces questions ne nécessitent pas d'écrire de code ni d'utiliser d'ordinateur : il vous faut résoudre ces exercices sur feuille.

D'autres questions vous demanderont d'implémenter des algorithmes en Python, et d'en mesurer la complexité par des méthodes empiriques de mesure de temps, qu'on appelle *profiling*. Il sera alors intéressant de comparer les résultats calculés sur papier avec ce qui est mesuré en pratique.

Exercice 1 : Manipulation des notations asymptotiques.

Donner les (meilleures) notations asymptotiques O des fonctions suivantes :

— $f(n) = 2 \times n^2 + 1000 \times n$
— $f(n) = n \times \log(n) + n^2$
— $f(n) = \log(n) + 2 \times \sqrt{n}$
— $f(n) = 2 \times 2^n + n \times 2^n$
— $f(n) = n \times (8 \times n + \sqrt{n})$
— $f(n) = n^3 + 3^n$
— $f(n) = n^2 \times \log(n) + n \times \log(n)^2$

Exercice 2 : Démonstration de propriétés notables.

Soient quatre fonctions f, g, ϕ et ψ de $\mathbb{N} \to \mathbb{R}_+$, telles que $f(n) \in O(\phi(n))$ et $g(n) \in O(\psi(n))$.

Question 1 Montrer qu'on a $f(n) + g(n) \in O(\phi(n) + \psi(n))$.

Question 2 Montrer qu'on a $f(n) \times g(n) \in O(\phi(n) \times \psi(n))$.

Exercice 3 : Déterminer la complexité théorique d'un algorithme.

Décrire ce que réalise l'algorithme 1. Calculer ensuite la complexité de l'algorithme (la fonction $f(n)$ décrivant le nombre d'*opérations élémentaires* réalisées avec une taille d'entrée n), puis déterminer la notation asymptotique O associée. Proposez une alternative à cet algorithme dont la complexité est en $O(1)$.

Algorithme 1

Entrée: $n \in \mathbb{N}^*$

```
k ← 0
pour i allant de 1 à n faire
    pour j allant de 1 à i faire
        k ← k + 1
    fin pour
fin pour
retourner k
```

Exercice 4 : Vérification empirique des complexités asymptotiques : profiling en Python.

Dans cet exercice, vous allez observer empiriquement (c'est à dire, par l'expérience) la complexité asymptotique de l'algorithme de la recherche dichotomique, que vous avez vu en cours.

Algorithme 2 Recherche dichotomique

Entrée: T un tableau de nombres triés, x l'élément recherché

```
deb ← 0
fin ← n
tant que deb ⩽ fin faire
    mid ← ⌊(deb+fin)/2⌋
    si x < T[mid] alors
        fin ← mid
    sinon si T[mid] < x alors
        deb ← mid + 1
    sinon retourner mid
    fin si
fin tant que
```

Question 1 Implémentez en Python l'algorithme 2, sous la forme d'une fonction `recherche_dichotomique`, prenant en paramètre une liste Python de nombres (`int` ou `float`) `T` et une valeur à recherche `x`.

Question 2 Ecrire une fonction en Python nommée `generer_tableau_trie` prenant en paramètre un entier naturel non-nul `n`. Cette fonction retournera une liste Python de nombres (`int` ou `float`) de longueur `n` triée par ordre croissant.

Question 3 Ecrire une fonction Python nommée `generer_valeur_a_chercher` prenant en paramètre une liste Python de nombres (`int` ou `float`) nommée `T`. Elle devra retourner un élément choisi aléatoirement au sein de la liste.

Vous pourrez vous aider de la bibliothèque Python `random`, en particulier de la fonction `choice()` qui prend en paramètre une liste et retourne un élément choisit aléatoirement au sein de celle-ci.

Question 4 En utilisant la fonction `time` de la bibliothèque `time` (qui retourne l'heure en secondes à un moment précis de l'exécution d'un morceau de code), mesurez le temps que prend l'exécution de la recherche dichotomique pour différentes tailles de tableaux. Pour chaque taille de tableau, on répetera la mesure un nombre important de fois (par exemple, $1,000$ ou $10,000$), et on conservera la moyenne de ces mesures pour représenter chaque taille de tableau considérée.

Question 5 Ensuite, écrivez du code Python affichant la courbe des temps mesurés en fonction de la taille du tableau. Affichez une deuxième courbe correspondant à la complexité asymptotique vue en cours. *On utilisera pour cela la bibliothèque* `matplotlib` .